A High-Speed FPGA-Based Hardware Implementation for Leighton–Micali Signature

Yifeng Song, Xiao Hu, Jing Tian, *Member, IEEE*, and Zhongfeng Wang, *Fellow, IEEE*

***Abstract*— Due to the rapid progress made in quantum computers, modern cryptography faces great challenges. Many digital signature schemes that have resistance to quantum computing are studied and standardized by several influential international organizations. The Leighton-Micali signature (LMS) protocol, one of the hash-based signature schemes, is standardized by both the Internet Engineering Task Force (IETF) and the National Institute of Standards and Technology (NIST) due to its well-studied security and relatively small signature size. However, the heavy computation load and high latency of LMS limits its practical applications. In this paper, for the first time, we propose a full hardware implementation of LMS to accelerate all the three stages: *key generation*, *signature generation*, and *verification*. Considering the scalability requirement and the characteristic of the parameter sets of LMS, we extract the coarse-grained basic logic, a hash group, and build a reconfigurable architecture for all available parameters by carefully designing the parallelism degree while achieving low latency and high hardware utilization efficiency. Then, we devise a fusion architecture for *key generation* and *signature generation* based on the hash group module. Moreover, for the *signature verification* stage, we propose a separate architecture by applying the hash group module along with an efficient depth-first Merkle tree module. We code our designs with Verilog language in parameterized style and implement them on a Xilinx XCVU7P FPGA platform. The experimental results show that significant improvements are obtained for different parameter sets by the proposed designs when compared to state-of-the-art works.**

***Index Terms*— Post-quantum cryptography (PQC), hash-based signature, Leighton-Micali signature, high-speed, hardware, FPGA.**

I. Introduction

In RECENT years, much progress has been made in quantum computers. Since 2018, IBM Q [1], IonQ [2], and Rigetti [3] have served as quantum cloud computing servers for scientific research. In 2021, SpinQ Technology announced the first two-bit desktop quantum computer, aiming to provide real-device experiments for education purposes [4]. It weighs 55kg with a dimension of $70 \times 40 \times 80$ cm^3, which is not far different from a traditional PC. Powerful quantum computers may run into our lives much sooner than we expect. By then, most widely-used public-key cryptographic schemes would be broken, including Rivest-Shamir-Adleman (RSA), Digital Signature Algorithm (DSA), and Elliptic Curve Digital Signature Algorithm (ECDSA) [5]. The security of these algorithms relies on the hardness of integer factoring or discrete logarithms computing, which is quite difficult for traditional computers to solve. However, equipped with Shor's algorithm [6], quantum computers can calculate these mathematical problems in polynomial time. In this way, quantum computing would pull many systems under significant risks, such as the internet communication system, which means our information security would be greatly challenged.

To respond to the challenge of quantum computing, a new branch of cryptography, called Post-quantum Cryptography (PQC), has been created and raised tremendous attention. The U.S. National Institute of Standards and Technology (NIST) has been collecting and evaluating talented PQC schemes for the standardization process since 2016. By July 2022, a total of four candidate algorithms have been selected for standardization, and four additional algorithms will continue into the fourth round of evaluation [7]. The final standard drafts will be finished in 2023 or even later because the attacks on these algorithms are still under investigation [8]. With the need to resist the threat of quantum computers, some devices that will be deployed recently and used for decades cannot wait long. Fortunately, the Internet Engineering Task Force (IETF) has standardized two hash-based signature (HBS) schemes for PQC applications, namely the Leighton-Micali Signature (LMS) [9] and the eXtended Merkle Signature Scheme (XMSS) [10]. NIST also approved the applications of these two algorithms with some recommendations for the parameter sets [11]. To promote the application of PQC, the IETF also released several proposals that attempt to introduce PQC algorithms into internet protocols like TLS (Transport Layer Security) and IKE (Internet Key Exchange) [12], [13], [14], [15], [16]. Studies show that more than 60% of internet connections are implemented over the TLS-based secure HTTPS (Hyper Text Transfer Protocol over SecureSocket Layer) protocol [17], [18]. However, related internet applications are facing significant challenges due to the computation and signature size overheads of PQC. Taking the PQC-based TLS as an example, [19] shows that around 70 requests are needed to be performed by clients per page on average, with 1-2KB resource fetched per request. The delay accumulated by each PQC computation will significantly affect the performance of the whole HTTPS protocol. Therefore, it will

Manuscript received 29 April 2022; revised 24 July 2022 and 31 August 2022; accepted 23 September 2022. Date of publication 3 October 2022; date of current version 25 January 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62104097, in part by the Key Research Plan of Jiangsu Province of China under Grant BE2022098, and in part by the High-Level Personnel Project of Jiangsu Province under Grant JSSCBS20210034. This article was recommended by Associate Editor C. H. Chang. *(Corresponding authors: Jing Tian; Zhongfeng Wang.)*

The authors are with the School of Electronic Science and Engineering, Nanjing University, Nanjing 210023, China (e-mail: yfsong@smail.nju.edu.cn; huxiao@smail.nju.edu.cn; tianjing@nju.edu.cn; zfwang@nju.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCSI.2022.3210016.

Digital Object Identifier 10.1109/TCSI.2022.3210016

1549-8328 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.

help a lot for PQC applications by exploring the high-speed acceleration schemes.

Although the theories of HBS have existed for decades, they did not attract much attention until the threat from the quantum computer became more realistic in recent years [20]. In the early stage, acceleration schemes are mainly implemented on software platforms [21], [22], [23]. [22] presents both LMS and XMSS implementations on an ARM Cortex-M4 platform. As for embedded system schemes, [24] proposed a software implementation of XMSS on a 16-bit smart-card. It shows the feasibility of XMSS in the constrained device but without a satisfying speed. [25] designed a hardware-software architecture to accelerate XMSS in constrained IoT motes, which achieves a 5.35x speed-up over an advanced software implementation. RISC-V SoC platform also attracts some attention. In [26] and [27], the authors provide their RISC-V-based solutions for XMSS acceleration respectively, both of which only implement part of the algorithm in hardware. Full hardware implementation has attracted more interest in recent years to obtain faster speed and better computational efficiency. In [28], the authors proposed an acceleration scheme of LMS for secure boot applications, in which only the signature verification function is designed in hardware. [29] presents the first full hardware implementation of XMSS, which shows 20% and 50% speedup over the work of [26] in *key generation* and *signature generation* phases, respectively. However, for high-speed applications, both LMS and XMSS still have much potential to be explored.

However, there are some feature differences between LMS and XMSS. According to the comparison of the two algorithms in [30], LMS is around 4 times faster than XMSS at the key and signature generation stages and about 5 times faster at the verification stage. As for the security, LMS has been proven secure in the Random Oracle (RO) model, while XMSS has been proven secure in the standard model [30]. However, IETF and NIST think both LMS and XMSS are safe enough for practical applications and approve their usage. Therefore, aiming at high-speed application scenarios and considering the runtime advantage of LMS, we focus on LMS and explore the feasibility of applying parallel processing architecture in the hardware implementation of LMS. We deliver a high-parallel and high-speed hardware implementation for the *key generation* stage of LMS that makes good use of the inherent parallelism of LMS and also balances the scalability for different parameter sets. This basic idea has been preliminary presented in our conference paper [31].

In this paper, we also focus on the high-speed implementation of LMS based on hardware platforms. First, We select the optimal hash algorithm in latency from existing powerful hash algorithms. Primarily, we theoretically analyze the operations of the SHA256 and SHAKE256-256 (the fixed 256-bits output size version of SHAKE256) algorithms and compare their latency and throughput in formulas. Then, based on the selected hash function, we present the first full hardware LMS design with significant improvements in latency, speed, and area efficiency compared with existing designs.

Our key contributions are summarized as follows:

- We analyze and compare the theoretical performance of SHA256 and SHAKE256-256, which are recommended for LMS in [11]. Moreover, we select SHAKE256-256 for our design based on the throughput analysis.
- For the first time, we propose a full hardware implementation of LMS to accelerate all the three stages: *key generation*, *signature generation*, and *verification*.
  1) We formalize the core operations in LMS and propose the corresponding computing module that can be used in all the three stages of LMS. This module, named hash group, consists of a group of parallelizing hash submodules.
  2) To take advantage of the characteristics of the parameter sets and take account of the hardware architecture scalability, we adopt a 34 parallelism architecture for the hash group module. This module shows a high-throughput performance and is adaptable to all parameter sets by changing the configuration values.
  3) We devise a fusion architecture that supports both *key generation* and *signature generation* by reusing the hash group module.
- We code our design in Verilog language and test it with all supported parameter sets. The implementation results on a Xilinx XCVU7P FPGA platform show that significant improvements are obtained by our design when compared to the state-of-the-art works.

The rest of the paper is organized as follows. Section II gives a brief introduction to LMS. Section III analyze and compare two hash algorithms recommended by NIST in [11]: SHA-256 and SHAKE256. Section IV shows our hardware architectures in detail. Section V and section VI are the experiment results and comparisons with existing works, respectively. Finally, Section VII concludes this paper.

II. Preliminaries

LMS is a kind of stateful HBS that uses a secure hash function to construct quantum-resisting signatures. The security of LMS depends only on the security of the underlying hash function [11]. As shown in Fig. 1, it consists of a Merkle tree and quantum-robust One-Time Signature (OTS) hash chains. "Stateful" means that each private key can be used only once, and the number of available private keys is decided by the tree height.

A. Merkle Tree

The Merkle tree in LMS is used to produce the public key and verify the signature in a safe and efficient way. In the *key generation* procedure, as shown in Fig. 1, the leaf nodes in the tree are firstly produced from the OTS private keys through the hash chains. Then, every two adjacent leaves are hashed to get the first layer of the internal nodes of the tree. These internal nodes are also compressed in such a way until the root node of the tree is calculated. This means that all the OTS private keys in LMS share the same public key, denoted as the red pentagon in Fig. 1.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.

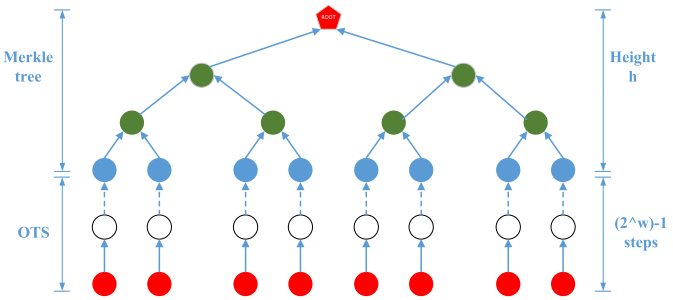


Fig. 1. The Merkle tree and the OTS hash chains of LMS. The red circles at bottom are the OTS private keys. The blue circles are leaf nodes of the Merkle tree. They are computed from the OTS public keys which are computed from OTS private keys by calculating the hash chains. The green circles are the internal nodes of the Merkle tree. The root of the Merkle tree is the public key of LMS, which is represented by the red pentagon.

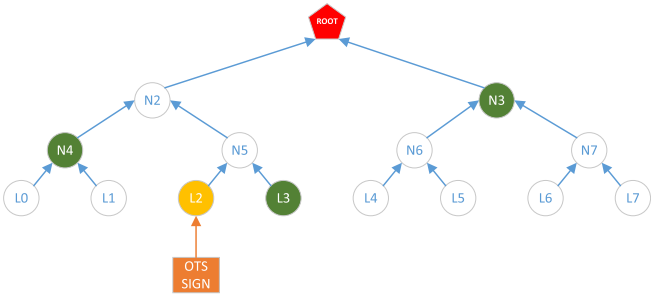


Fig. 2. The authentication path of Merkle tree. The yellow circle is where the currently using OTS is located. The green circles are the nodes needed to be recorded in the authentication path. The LMS public key represented by the red pentagon should also be included.

The LMS signature is formed with the OTS signature, the internal nodes in the path, and the LMS public key. In the *signature generation* procedure, the OTS *signature generation* is executed first. Then, all the nodes in the authentication path should be calculated or read from memory. In the *signature verification* procedure, the OTS signature is first verified to obtain the corresponding leaf node. Then, the leaf node is used to calculate the internal nodes along the authentication path. In this way, a new root will be generated at last. If this root is the same as the public key announced by the signer, the signature will be verified as "Accept"; otherwise, as "Reject".

Figure 2 shows an example of the authentication path in LMS. The L2 in yellow is where the current OTS is located. So, the nodes L3, N4, N3, and the root are all combined with the OTS signature (the orange block in the figure) to form the LMS signature. To verify the signature, the OTS verification is firstly executed to get a temporary L2. The node L3 and the temporary L2 are hashed to get a temporary N5, which is combined with N4 to get a temporary N2. At last, the temporary N2 and the node N3 form a new root, which will be compared with the public key to determine the verification result.

B. One-Time Signature (OTS)

LMS uses the Winternitz one-time signature (WOTS) [32] as its OTS scheme, which is designed to use one string in the key to simultaneously sign several bits in the message digest. Firstly, there are several parameter sets of LMS, which are denoted with some symbols. Table I shows the notations for the parameter symbols used in LMS. n is the number of bytes of the output of the hash function. w is the width (in bits) of the Winternitz coefficients, which presents a space/time trade-off: increasing the value of w will shorten the signature length while key generation and verification will take more time to finish. p is the number of Winternitz chains used in a single WOTS operation, which is a function of n and w. ls is the number of left-shift bits used in the checksum function $Cksm$. h is the height of a single Merkle tree. Fig. 3 shows the three core algorithms of the WOTS scheme and the relations among them. Some insignificant details are omitted in this figure for brevity. For a more detailed description of the WOTS algorithms in LMS, please refer to the document offered by the IETF [9].

TABLE I
MEANINGS FOR PARAMETER SYMBOLS USED IN LMS

Symbols	Meaning
n	The length of the hash output (in bytes).
w	The width (in bits) of the Winternitz coefficients; that is, the number of bits from the hash or checksum that is used with a single Winternitz chain.
p	The number of Winternitz chains used in a single WOTS operation. This is a function of n and w.
ls	The number of left-shift bits used in the checksum function Cksm.
h	The height of a single Merkle tree.

In the WOTS *key generation* procedure, a random 16-byte string I and a random 32-byte string $SEED$ are firstly generated at the same time. Then, for the q-th WOTS node, I, q, the array index i, the fixed 2-byte value 0xff, and $SEED$ are combined together and hashed to get the array element $x[i]$. The array x is the kernel of the private key, which is used to generate both the public key and the signature. For the WOTS public key, a hash chain is applied to each array element $x[i]$. I, q, array index i, and loop index j are mixed with the intermediate variable $temp$ in each iteration. $temp$ firstly equals $x[i]$ and then stores the output value of the hash function in each round. When the number of iterations comes to 2^w, the hash outputs are saved into the array y. At last, a final hash is performed on I, q, the fixed value D_PBLC (0×8080), and the array y to get the value K, which is the key material of the leaf node in the Merkle tree.

Algorithm 1 Checksum Calculation (Cksm)

Input: n-byte String S, n, w, ls
Output: 16-bit unsigned integer $Cksm_out$
1: Set $sum = 0$
2: **for** $i = 0; i < (n * 8/w); i = i + 1$ **do**
3: $sum = sum + (2^w - 1) - coef(S, i, w)$
4: Return string: $sum \ll ls$
5: **end for**

Algorithm 1 shows the checksum calculation algorithm, which is performed in both signing and verifying processes.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.

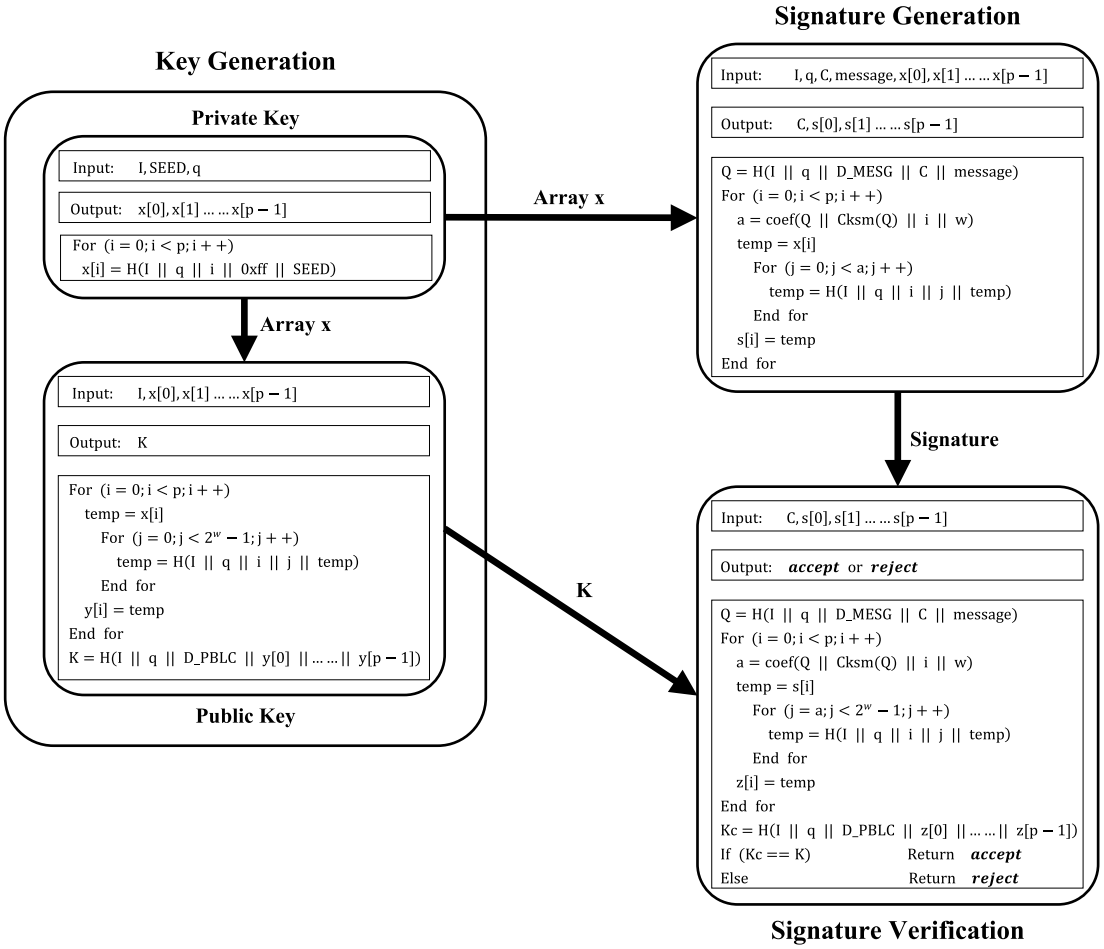


Fig. 3. The three main algorithms of WOTS scheme,including *Key Generation*, *Signature Generation*, and *Signature Verification*.

This function is used to detect the forgery attempt of manipulating the elements of the signature. $coef(S, i, w)$ is a function to pick up the i-th, w-bit value of the string S. For example, if S is the string 0001001000110100, then $coef(S, 7, 1)$ is 0 and $coef(S, 0, 4)$ is 0001.

In the WOTS *signature generation* procedure, an n-byte random string C is generated first. After that, a hash function is applied to I, q, the fixed value D_MESG (0×8181), C, and the message string to obtain the digest value Q. Then, a $coef$ function nested with a checksum function inside is applied to get the loop number a for each $x[i]$. By calculating a hash chain which is similar to the one in the public key generation, the signature array s can be obtained. The random string C along with the array s forms the signature of the message.

The verification procedure can be regarded as a complementary operation of the hash chain in the signing process. Part of the hash chain, from 0 to $a - 1$, is performed in the signature generation procedure. When verifying the signature, the rest of the hash chain, from a to $2^w - 1$, is finished using the array s. The result array z will be equal to the array y in the public key generation, if the signature is not tampered. To simplify the checking process, a hash operation is executed to get the test value Kc. If Kc is equal to the public key K, the signature and the message will be accepted; otherwise, they will be rejected.

TABLE II
SECURITY STRENGTHS OF THE SHAKE256-256 AND SHA-256

Function	Output Size	Security Strengths in Bits		
		Collision	Preimage	2nd Preimage
SHA-256	256	128	256	$256 - L(M)$*
SHAKE256	d	$min(d/2, 256)$	$\geq min(d, 256)$	$min(d, 256)$

* For a message M, the function $L(M)$ is defined as $\log_2(len(M)/B)$, where B is the block length of the function in bits, $B = 512$ for SHA-256.

III. SHAKE256-256 Vs SHA-256

SHA-256 is a member of the SHA-2 family specified in [33], while SHAKE256 is a member of the SHA-3 family specified in [34]. More specifically, SHAKE256 is an extendable-output function (XOF) in which the output can be extended to any desired length. However, according to the recommendations in [11], the output length of SHAKE256 is fixed to 256 in this work, denoted as SHAKE256-256. Both SHA-256 and SHAKE256-256 have input block sizes bigger than 55-byte, compress messages into a fixed 256-bit digest, and are recommended in LMS by NIST [11]. Table II shows the security strength of SHA-256 and SHAKE256 summarized in [34]. For SHAKE256-256, d equals 256, which means SHAKE256-256 and SHA-256 have the same security strength against collision and preimage attacks. In LMS,

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.

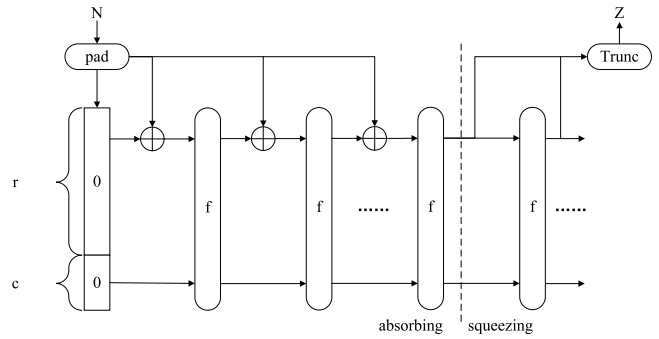


Fig. 4. The sponge structure of SHA-3. It absorbs the message in fixed width pieces and squeezes the digest out also in fixed size.

$len(M)$ always equals 440 or 688 when the length of the hash output $n = 32$, which leads to that the security strength against second preimage attacks of SHA256 and SHAKE256-256 are almost the same. As mentioned in Section II, the security of LMS depends only on the security of the underlying hash function [11]. Therefore, SHAKE256-256 is as safe as SHA256 to construct LMS. In the following subsections, we discuss and compare them based on throughput performance in detail to decide the optimal one for our hardware implementation.

A. SHAKE256-256

All the algorithms in the SHA-3 family share the same structure called sponge structure. As shown in Fig. 4, the arbitrary length message N is firstly padded according to the rule that the message after padding should be an integer multiple of r. The string, which is with the length of $r + c$ and initialized to 0, is called state in the sponge structure. The padded message is divided into pieces and absorbed by the permutation function f to update the state. After all of the message is absorbed, the output starts to be squeezed out in fixed size. For SHAKE256, the parameters mentioned above can be referred in [34].

[!t] **Algorithm 2** Permutation Function f (KECCAK)

Input: State array A
Output: New state array A'
1: **for** $i = 0; i < 24; i = i + 1$ **do**
2: **Theta step** : $(0 \le x, y \le 4, 0 \le z < 25)$
$C[x, y, z] = A[x, 0, z] \oplus A[x, 1, z] \oplus A[x, 2, z] \oplus A[x, 3, z] \oplus A[x, 4, z]$
$D[x, z] = C[(x - 1), z] \oplus ROT(C(x + 1), 1)$
$A'[x, y, z] = A[x, y, z] \oplus D[x, z]$
3: **Rho step** : $(0 <= x, y \le 4)$
$A[x, y, z] = ROT(A'[x, y, z], r[x, y])$
4: **Pi step** : $(0 \le x, y \le 4, 0 \le z < 25)$
$B[y, (2x + 3y), z] = A[x, y, z]$
5: **Chi step** : $(0 \le x, y \le 4, 0 \le z < 25)$
$A[x, y, z] = B[x, y, z] \oplus (NOT(B[x + 1], y, z)) AND(B[(x + 2), y, z])$
6: **Iota step** : $(0 \le z < 25)$
$A'[0, 0, z] = A[0, 0, z] \oplus RC[z]$
7: **end for**
8: Return state A'

The permutation function f processes the state for 24 rounds. Each round consists of five steps: $Theta$, Rho, Pi, Chi, and $Iota$, which are shown in Alg. 2. The XOR($\oplus$), NOT, and AND are bit-wise logical operations, while ROT is a left circular shift by $r[x, y]$ bits and RC is a round constant array.

B. SHA-256

The procedure of SHA-256 is quite similar to SHA-3 in some aspects. The input message is also padded and divided into fixed-length pieces, which are then fused with each other through the permutation function Map. Alg. 3 shows the processing steps in the Map function. The i-th message segment is represented as a 16 elements array, which is extended to a 64-element array by using the original elements to compute the supplement elements. Then, in 64 rounds of iterations, this new array is applied to update the intermediate values from $H^{(i-1)}$ to H^i. After all the message pieces are involved, the last updated array H is output as the digest string. Six fundamental functions are described as follow:

$$Ch(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z) \tag{1a}$$

$$Maj(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) \tag{1b}$$

$$\sum_{0}^{256}(x) = ROTR^{2}(x) \oplus ROTR^{13}(x) \oplus ROTR^{22}(x) \tag{1c}$$

$$\sum_{1}^{256}(x) = ROTR^{6}(x) \oplus ROTR^{11}(x) \oplus ROTR^{25}(x) \tag{1d}$$

$$\sigma_0^{256}(x) = ROTR^{7}(x) \oplus ROTR^{18}(x) \oplus SHR^{3}(x) \tag{1e}$$

$$\sigma_1^{256}(x) = ROTR^{17}(x) \oplus ROTR^{19}(x) \oplus SHR^{10}(x) \tag{1f}$$

All the values in these functions, which are represented as x, y, and z, are 32-bit words. $\neg$, $\oplus$, $\vee$, and $\wedge$ are bit-wise logical operations, which denote complement operation, XOR, OR, and AND respectively. SHR is the right shift operation and $ROTR$ is the right-rotation operation.

C. Analysis and Selection of the Two Hash Algorithms

Now we analyze the throughput performance differences between SHAKE256-256 and SHA-256. Due to the serial feature of the hash chain in LMS, some widely-used throughput optimization methods like unrolling with pipelining used in [35] do not work in this situation because of the lack of continuous input streams. Therefore, only the basic iterative architecture is considered and analyzed in this work. In this way, we can evaluate the throughput of this kind of structure by taking into account the input block size B, the iteration number I, and the critical path delay D in each iteration, which is shown in Table III. And the throughput TP is calculated by: $TP = \frac{B}{I \times D}$. For SHAKE256-256, the absorbed block size is 1088 bits according to the rule in [34] while the block size in SHA-256 is 512 bits shown in [33]. The iteration numbers of

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 Permutation Function Map

Input: Array $H^{(i-1)}$, message $M^{(i-1)}$
Output: New array H^i
1: Compute the message schedule, W_t

$$W_t = \begin{cases} M_t^{(i)} & 0 \le t \le 15 \\ \sigma_1^{256}(W_{t-2}) + W_{t-7} & \\ +\sigma_0^{256}(W_{t-15}) + W_{t-16} & 16 \le t \le 63 \end{cases}$$

2: Initialize the working variables a, b, c, d, e, f, g,and h, with the $(i-1)^{st}$ hash value:
$a = H_0^{(i-1)}$, $b = H_1^{(i-1)}$, $c = H_2^{(i-1)}$, $d = H_3^{(i-1)}$, $e = H_4^{(i-1)}$, $f = H_5^{(i-1)}$, $g = H_6^{(i-1)}$, $h = H_7^{(i-1)}$
3: **for** $t = 0; t < 64; t = t + 1$ **do**
4: $T_1 = h + \sum_1^{256}(e) + Ch(e, f, g) + K_t^{256} + W_t$,
$T_2 = \sum_0^{256}(a) + Maj(a, b, c)$,
$h = g$, $g = f$, $f = e$, $e = d + T_1$,
$d = c$, $c = b$, $b = a$, $a = T_1 + T_2$
5: **end for**
6: Compute the ith intermediate hash value $H^{(i)}$
$H_0^{(i)} = a + H_0^{(i-1)}$, $H_1^{(i)} = b + H_1^{(i-1)}$, $H_2^{(i)} = c + H_2^{(i-1)}$, $H_3^{(i)} = d + H_3^{(i-1)}$, $H_4^{(i)} = e + H_4^{(i-1)}$, $H_5^{(i)} = f + H_5^{(i-1)}$, $H_6^{(i)} = g + H_6^{(i-1)}$, $H_7^{(i)} = h + H_7^{(i-1)}$
7: Return array H^i

TABLE III
COMPARISON BETWEEN SHAKE256-256 AND SHA-256

Hash type	Block size	Data width	Iterations	Critical path
SHAKE256-256	1088 bits	1600 bits	24	7 XORs + 1 NOT + 1 AND
SHA-256	512 bits	32 bits	64	2 XORs + 7 32-bit adders

these two algorithms are equal to 24 and 64 respectively. This means that the throughput of SHAKE256-256 is $\frac{1088}{512} \times \frac{64}{24} = 5.67$ times of that of SHA-256 if they work at the same clock frequency. But the critical path of SHAKE256-256 consists of 7 XORs, 1 NOT, and 1 AND, while there are 2 XORs and 7 32-bit adders in the critical path of SHA-256. It is very clear to see that SHAKE256-256 has a shorter critical path than SHA256 which leads to a higher maximum clock frequency. Therefore, we choose SHAKE256-256 to implement the high-speed LMS accelerator for its obvious advantage in throughput. In the following discussion, we will see that the hash module would heavily impact on the clock frequency of the proposed overall designs.

IV. PROPOSED ARCHITECTURES

In the proposed architectures, we realize the iterative hash operations with a high-throughput hash group module, and implement the Merkle tree an optimized depth-first computing module. For the common server/client application model, the server usually executes the key and signature generation while the client verifies the signature. Moreover, the key generation and the signing process always happen at the same place. Therefore, based on these two modules, we propose a full hardware implementation for LMS, which is divided into two parts: one combines *Key Generation* and *Signature Generation*, denoted as *Key_gen&Sign*; the other is *Signature Verification*.

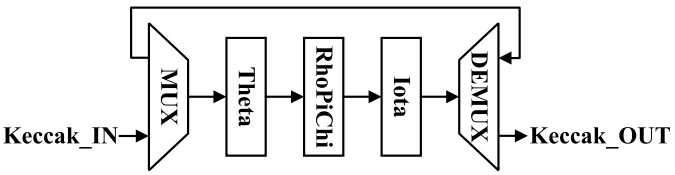


Fig. 5. KECCAK module: Architecture of the permutation function in SHAKE256-256.

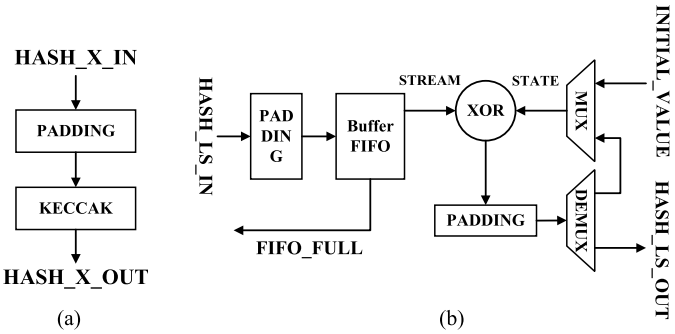


Fig. 6. (a) HASH_X module: SHAKE256-256 with input that is shorter than 1088-bits. (b) HASH_LS module: SHAKE256-256 with input that is longer than 1088-bits.

TABLE IV
COMPARISON WITH EARLY WORKS OF SHA3 AND SHA-256

Work	Hash Type	Device	F_{max} (MHz)	Area (Slices)	Throughput (Gbps)
[36]	SHA-256	Virtex-6	354	197	1.41
[37]	SHA-256	Virtex-6	172	1831	11.00*
[35]-Case V	SHA3-512	Virtex-6	344	1406	16.51*
[35]-Basic	SHA3-512	Virtex-6	153	871	3.68
This work	SHAKE256-256	Virtex-7	294	1421	12.80

* [37] and [35]-Case V are realized by using unrolling with pipelining method to improve the throughput and their throughput results are all calculated under the conditions of data pipeline, which is not suitable for LMS design.

A. Hash Module

1) Hardware Architecture: The hash module is the core function of LMS. Fig. 5 shows the architecture of the permutation function in SHAKE256-256. Two kinds of hash modules used in our design are shown in Fig. 6. For the input strings whose width equals X and $X < 1088$, the SHAKE256-256 function is computed by the *HASH_X* module, consisting of a single *KECCAK* module along with a small padding module. In our design, *HASH_440* ($X = 440$) and *HASH_688* ($X = 688$) are mainly used. And for the input strings that are longer than 1088 bits, the function is designed as the *HASH_LS* module where a FIFO is needed to cache and allocate the input into 1088-bit streams as inputs for each round of *KECCAK*.

2) Experiment Comparison: As shown in Table IV, we compare our SHAKE256-256 implementation with related

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.

TABLE V
LMS PARAMETER SETS FOR SHAKE256_256

LMS Parameter Sets	n	w	p	sig_len
LMS_SHAKE_N32_W1	32	1	265	8516
LMS_SHAKE_N32_W2	32	2	133	4292
LMS_SHAKE_N32_W4	32	4	67	2180
LMS_SHAKE_N32_W8	32	8	34	1124

works [35], [36], [37] of SHA3 and SHA256. All these works are based on Xilinx Virtex FPGA platforms. [36] and [37] are two implementations of SHA-256 aiming at high-speed and low-cost, respectively. [36] adopts the folded architecture to reduce resource cost, which requires 129 clock cycles to finish the computation. [37] adopts the unrolling architecture with several stages of pipelines inside, which can improve the throughput performance with adequate input data streams. [35] presents the basic iterative and several optimized implementations for SHA3-512. It is shown in Table IV that the throughput performance of SHA256 in [36] is significantly lower than our design of SHAKE256-256, which is in accord with our analysis in Section III-C. The advantage of F_{max} in [36] is mainly due to the folded architecture and the lower net delay caused by the smaller data width compared to SHAKE256-256. The Case V design in [35] and the SHA256 design in [37] show good performances in throughput, which are close to that of our design. But both of them are realized by using unrolling with pipelining method to improve the throughput and their throughput results are all calculated under the conditions of sufficient input data, which, however, is not applicable to the proposed LMS design. If this kind of architecture is applied in the hash module of our LMS design, the actual throughput of the hash module would be half or even a quarter of the anticipative result due to the lack of continuous input streams. As for the basic design in [35], our design of SHAKE256-256 outperforms it in both F_{max} and throughput performance.

B. Parallel Hash Group

According to the algorithm described in Fig. 3, iterative hash operations are the core computation tasks that are performed in all three stages of LMS. And these computation tasks can be divided into p individual tasks, which is quite suitable for hardware acceleration with parallel processing scheme. In this paper, we use the parameter sets for SHAKE256_256 which is recommended in [11]. As shown in Table V, there are four different values for p: $34, 67, 133$, and 265. We find that the value of p can be represented as a formula of w: $p = 33 * (8/w) + 1$, where $w = 8, 4, 2, 1$, respectively. But if we straightforwardly adopt the degree of parallelism as 33, one more hash loop should be performed for all parameter sets, and more than 96% hash modules are left unused in the last round. It even doubles the computation time when p equals 34. Therefore, we slightly transform the equation as $p = 34 * (8/w) - (8/w - 1)$ and set the degree to 34 in our design. In this way, we use 34 hash modules to construct a parallel hash group, which can achieve low latency and high hardware utilization efficiency at the same time. Except for the last round, all the hash modules are fully utilized. In the last round, the hardware utilization is 100% for $w = 8$; 97% for $w = 4$; 91% for $w = 2$; 79% for $w = 1$. Moreover, it is scalable and can be adapted to all the parameter sets. For $p > 34$, this module would be executed many times to obtain all the array elements. The loop number constantly equals $2^w - 1$ for each hash function in the $Key\ Generation$ procedure. But in the $Signature\ Generation$ procedure, each loop number in the hash group does not have same value due to the features of the coefficient calculation. Targeting at constant time, every hash module keeps working until all the hash loops are finished. And the right results of these modules are output at the same time. Similarly, for $p > 34$, some hash modules do not have computing tasks at the last round. We also activate these hash cores to resist the timing attacks.

TABLE VI
EXTERNAL MEMORY SIZE NEEDED FOR LMS WITH DIFFERENT h

h	5	10	15	20
Memory size	16kb	512kb	16Mb	512Mb

C. The Depth-First Merkle Tree Implementation

The nodes of the Merkle tree are used to form the authentication path of the signature. Aiming at high-speed applications especially in the server/client model, all of the nodes generated in the key generation procedure are stored in the external memory, and then parts of them are accessed in the signing process. The external memory sizes needed for LMS with different h are summarized as shown in Table VI, ranging from 16kb to 512Mb. The data reading and writing are achieved by the DDR4 interface. In this way, the Merkle tree does not need to be recomputed and the short reading latency of DDR4 replaces the huge recomputing latency. As shown in the right part of Fig. 8, the output array of the hash group is stored in the FIFO connected to the *HASH_LS* module, which is used to get the value of K of the OTS public key. A *HASH_440* module is implemented to obtain the Merkle tree's leaf nodes. Then, as shown in Fig. 7, we use h 256-bit registers to form a memory stack for the tree. Each register temporarily stores the latest computed node at each layer of the tree. This trick can avoid reading from the external memory with a small resource cost. The even nodes at each layer are stored in the corresponding registers through the demultiplexer. When the followed odd node is obtained, the even leaf in the register is mixed with the odd leaf to compute the father node at the next layer. It can be summarized that each leaf node takes $\{1 + p + (p * 256)/1088 + 1\} f(KECCAK)$ operations and the worst case for obtaining a new leaf node are $\{h - 1\}$ $f(KECCAK)$. It should be pointed out that computing task congestion would never happen.

D. Key_Gen&Sign

The *Key_gen&Sign* module is a fusion module of $Key\ Generation$ and $Signature\ Generation$, which can perform these two functions by adjusting the configurations.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.

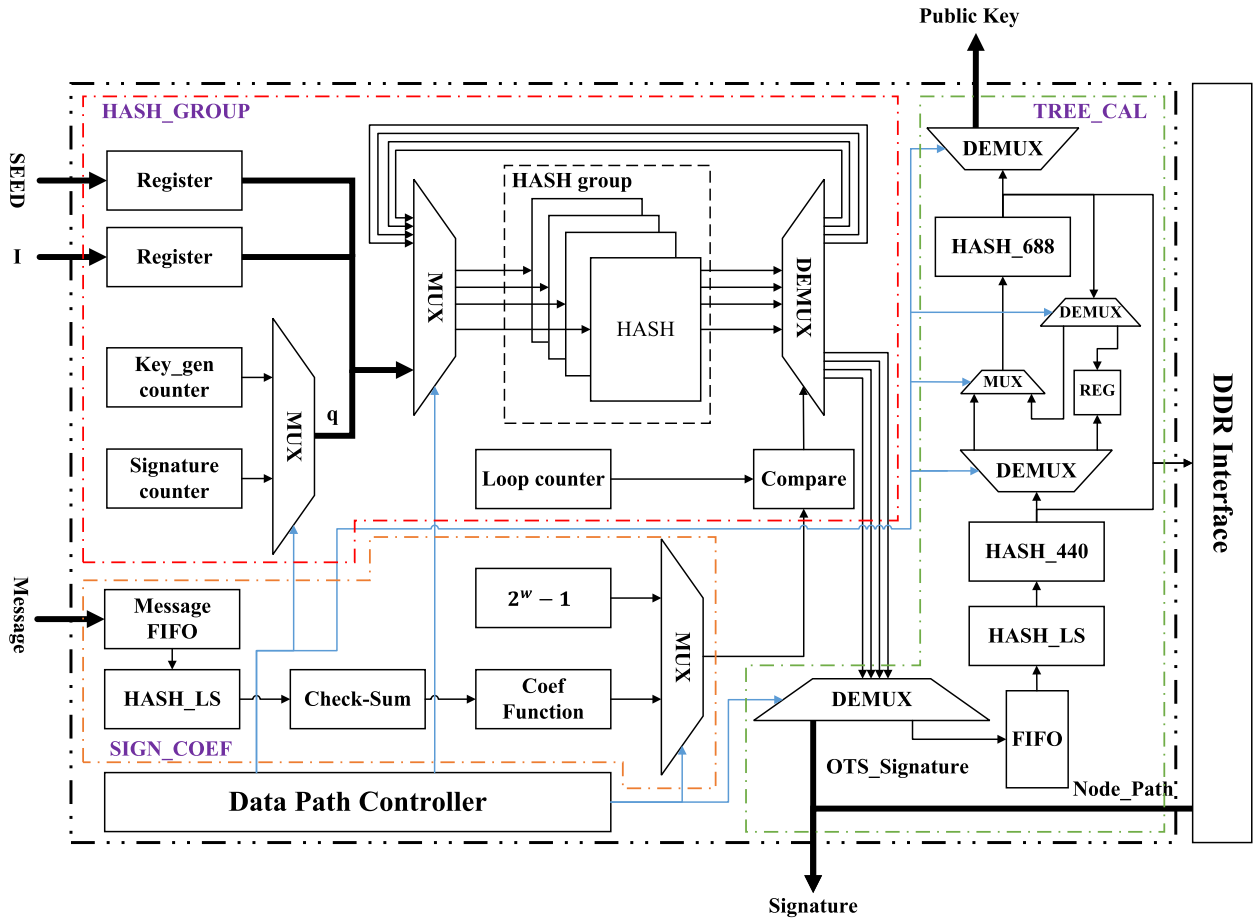


Fig. 7. The fusion architecture of *Key Generation* and *Signature Generation*. The hash group boxed by the red dotted line is reused in both functions. The private keys are generated in the first loop of the hash group, which do not have output ports and are not denoted in this diagram.

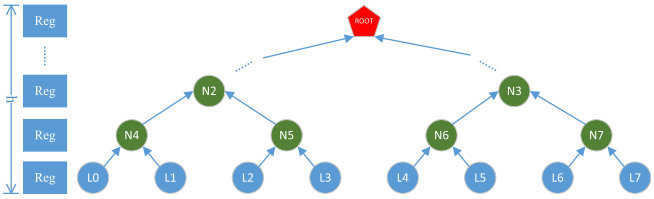


Fig. 8. The memory stack for the Merkle tree, which consists of h 256-bit registers corresponding each layer of the tree.

Figure 8 shows the top-level architecture of the *Key_gen&Sign* module, containing three parts. First, the circuits inside the red dotted line box are the reused hash group and two registers used to store the random values $SEED$ and I. And two counters to serve for *Key Generation* and *Signature Generation* respectively are also included. Second, on the red box's right side is the Merkle tree's calculation part. All the nodes in the tree are computed in this part and stored in DRAM through the DDR4 interface. The third part is below the red box, which are used in the signature generation stage. The message needed to be signed is hashed, and the 256-bit digest result is sent to compute the coefficients for the hash group.

During the LMS key generation, the random numbers $SEED$ and I are firstly obtained from the input ports and stored in the registers. Meanwhile, the counter for key generation is activated and output q to the hash group through a multiplexer controlled by the data path controller. Then, with the input data consisting of $SEED$, I, and q, the hash group performs 34 individual hash operations for p/34 rounds of iterations. There are 2^w hash operations in each hash iteration, in which the first hash generates the element of the OTS private key and the rest $2^w - 1$ hash operations produce elements of the OTS public key. Through the demultiplexer after the output ports of the hash group, all the elements of the OTS public key array are stored in the FIFO connected to the *HASH_LS* module. After that, the Merkle tree nodes are computed and stored in DRAM as described in the above subsection. The message is sent into the *Message FIFO* to compute the digest by the *HASH_LS* module for the signature generation. Then, the coefficients for the hash group are calculated by the *Check-Sum* module and the *Coef Function* module with the message digest. Controlled by the data path controller, the newly obtained coefficients and the q from the signature counter are transported to the hash module to calculate the OTS signature array. Meanwhile, the indexes of the nodes in the authentication path are sent to the DDR4 interface controller, and h 256-bit data are serially read from memory. The reading latency is relatively small and the nodes in the path can always be obtained before the OTS signature is finished.

E. Signature Verification

As shown in Fig. 9, the hash group with 34 degree of parallelism is also adopted in the *Signature Verification* architecture to make it fast and flexible. In this procedure, there

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.

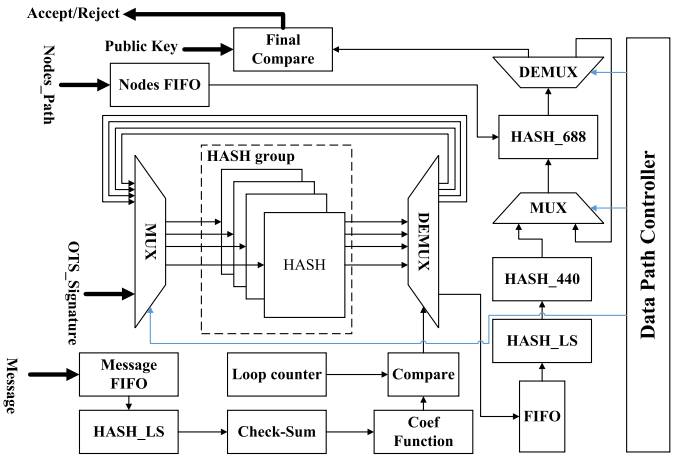


Fig. 9. The top architecture of the *Signature Verification*.

TABLE VII

RESOURCE UTILIZATION OF THE MAINLY MODULES OF LMS KEY GENERATION AND SIGNATURE GENERATION WITH PARAMETERS $w = 1$, $p = 265$, AND $h = 5$

Module	LUTs	FFs	BRAMs	DSPs	CLBs
HASH_GROUP	109427	72462	0	0	19441
SIGN_COEF	4962	6539	15.5	0	1118
TREE_CAL	16619	13028	17	0	6132
DDR4 IP	8348	10313	25.5	3	1913
Total	139886	103914	58	3	26342
Available	788160	1576320	1440	4560	98520
Percentage	17.82%	6.59%	4.03%	0.07%	26.7%

are several parts of input data for different purposes, including the message waiting for verification, the OTS signature with the nodes in the authentication path, and the public key preserved by the verifier. the input message is stored in a FIFO connected to the *HASH_LS* module, by which the digest of the message can be calculated and the the number of iterations for the hash group can also be obtained with the help of the *Check-sum* module and the *Coef Function* module. Similar to *Signature_Generation*, the hash modules are kept running until all the results are computed. Then one more *HASH_LS* and a *HASH_440* are used to calculate the temporary OTS public key K_c. The nodes in the authentication path are sent to a *HASH_688* module to recompute the LMS public key T_c using K_c. Finally, the public key preserved by the verifier is compared with T_c to decide to output *Accept* or *Reject*.

V. RESOURCE AND TIMING PERFORMANCE EVALUATION

The high-speed architecture proposed in this paper can be adopted to all the parameter sets by changing the configuration values. Without loss of generality, we take $\{w, p, h\} = \{1, 265, 5\}$ as an example case for resource evaluation of the hardware implementation. As for the timing performance experiments, we test all the parameters sets to get a comprehensive evaluation. Both *Key_gen&Sign* and *Verification* are coded in Verilog and implemented with Vivado 2018.3 on Xilinx Virtex UltraScale+ device (xcvu7p-flva2104-3-e), which are discussed respectively as follows.

TABLE VIII

RESOURCE UTILIZATION OF THE LMS VERIFICATION WITH PARAMETERS $w = 1$, $p = 265$, AND $h = 5$

Module	LUTs	FFs	BRAMs	CLBs
OTS_VERIFY	113817	55066	0	24382
VERIFY_COEF	4633	4671	15.5	1283
ROOT_CAL	11016	8607	17	2836
Total	129990	68610	32.5	28501
Available	788160	1576320	1440	98520
Percentage	16.49%	4.35%	2.26%	28.9%

A. Key_Gen&Sign

The *Key_gen&Sign* module consists of four main parts: HASH_GROUP, SIGN_COEF, TREE_CAL, and one DDR4 IP from XILINX. As shown in Table VIII, the whole *Key_gen&Sign* module consumes 139886 LUTs, 103914 FFs, and 58 BRAMs, which take 17.74%, 6.59%, and 4.03% of the available resources respectively. Containing 34 individual KECCAK modules, HASH_GROUP becomes the largest part of the entire design with 109427 LUTs and 72462 FFs. The TREE_CAL module has one SHAKE_LS submodule, two HASH_440 submodules, and one 1088-bit 64-depth FIFO, which consumes 16619 LUTs, 13028 FFs, and 17 BRAMs as the second large module. The DDR4 IP provided by Xilinx needs 8348 LUTs, 10313 FFs, and 25.5 BRAMs. And the SIGN_COEF mainly uses one SHAKE_LS to compute the coefficient needed in the signature generation, which is the smallest part in the design with 4962 LUTs, 6539 FFs, and 15.5 BRAMs. Due to the reasonable reuse of some submodules and the well-coded implementation, we merge the *key generation* and the *signature generation* functions into one module with even 26.7% fewer LUTs and 17.9% fewer FFs compared with our previous *key_generation* implementation in [31].

B. Signature Verification

The resource utilization of the *Signature Verification* module is shown in Table VII. Similar to the *Key_gen&Sign* module, there are three main parts: VERIFY_COEF to calculate the hash loop coefficients; OTS_VERIFY to compute the OTS test value K_c; and ROOT_CAL using K_c to get the test LMS public key for verification. The total resource consumption comes to 129990 LUTs, 68610 FFs, and 32.5 BRAMs, which take 16.49%, 4.35%, and 2.26% of the available resources, respectively.

C. Timing Performance

We evaluate the performance of our design by using the simulator of Vivado 2018.3. Limited by the user clock frequency of the DDR4 IP, the maximum clock frequency of the *Key_gen&Sign* is 267MHz. As for the *Signature Verification* module, the maximum clock frequency can reach at 294MHz. It should be pointed out that the proposed hash group lies in the critical path of the *Signature Verification* module and is the second longest data path of the *Key_gen&Sign* module,

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.

TABLE IX

NUMBER OF CYCLES FOR KEY GENERATION, SIGNATURE GENERATION, AND SIGNATURE VERIFICATION OF OUR DESIGN WITH DIFFERENT PARAMETERS

h	w	key_gen (clock cycles)	sig_gen (clock cycles)	verify (clock cycles)
5	8	2.22×10^5	7049	7327
	4	2.82×10^4	998	1553
	2	2.91×10^4	572	1516
	1	5.70×10^4	584	2384
10	8	7.09×10^6	7049	7461
	4	8.94×10^5	998	1687
	2	9.23×10^5	572	1650
	1	1.81×10^6	584	2517
15	8	2.27×10^8	7049	7594
	4	2.86×10^7	998	1820
	2	2.95×10^7	572	1783
	1	5.74×10^7	584	2651
20	8	7.26×10^9	7049	7727
	4	9.15×10^8	998	1953
	2	9.44×10^8	572	1916
	1	1.83×10^9	584	2784

whose critical path locates in the DDR4 IP. In the signature generation and verification test, we set the length of the message to sign as 4×1088 bits after padding, which takes 27×4 cycles for the SHAKE_LS module to compute the final digest. If the message needed to sign is $M \times 1088$ bits after padding, the signature generation and verification latency need to plus $(M-4) \times 27$ cycles, respectively. Table IX shows the numbers of cycles for key generation, signature generation, and signature verification of our design with different parameters. As expected, the number of cycles increases exponentially with h for the key generation stage that is the most time-consuming part among the three stages. Moreover, two points should be addressed from these experiments' results. The first one is that it takes the same number of cycles to sign the same message with different values of parameter h. The reason is that the operations of reading node values from DDR4 and executing the OTS verification procedure are processed simultaneously, and it takes much few cycles to read from memory. The second point is about the signature generation time. As mentioned before, the numbers of loops inside each hash iteration in the signature generation and verification are set as constant value in our design, which are 2^w for different w. And $8/w$ hash iterations are needed for each signing process. Therefore, the total delay of the signature generation is around $8/w \times 2^w$ hash function delay plus the message compression time (126 cycles in our test). After subtracting 126, the numbers of cycles for signature generation with different w in our test are nearly $8:2:1:1$, which is in accord with the previous analysis.

VI. COMPARISON WITH EXISTING IMPLEMENTATIONS

There are very few implementation works of LMS at present. Table XI lists all existing implementations of LMS on different platforms for several parameter sets, *i.e.*, $\{h, w\} = \{5, 8\}, \{5, 4\}, \{20, 8\}$, and $\{20, 4\}$. [38] is an open software implementation that is usually used as a baseline in various implementation works, which is tested on an Intel Xeon CPU 3.3GHz with threading enabled in our simulation. In [23], the authors implemented LMS on an STM32 ARM Cortex M4 platform. [31] and [28] are two state-of-the-art LMS hardware implementation works. [31] only implements the key generation stage of LMS on a Xilinx ZCU104 platform, while [28] only implements the signature verification stage on a Xilinx XCKU025 platform. In our design, we use external memory to store the Merkle tree, which can save the recomputing time of the tree nodes in the authentication path. To make a fair comparison, the signature generation time of [23] in Table XI is computed by subtracting the original signature generation time with the key generation time, which can be regarded as the pure signing time similar to the situation of our design. Since many previous works focused on XMSS have been presented, we also include two state-of-the-art implementation works of XMSS [26] and [29] in Table XI to make the comparisons more sufficient. [26] is a software-hardware co-design based on RISC-V. [29] is a full hardware implementation and is the fastest XMSS implementation up to now. In consistence with the notation conversing method used in [23], these two XMSS designs with $w = 16$ are compared to our design with $w = 4$. Our design shows obvious advantages in both clock frequency and computing delay.

TABLE X

SEPARATELY COMPARISON WITH THE STATE-OF-THE-ART WORKS FOR KEY GENERATION, SIGNATURE GENERATION, AND SIGNATURE VERIFICATION

Stage	Design	Parameters	Cost (LUTs/FFs/BRAMs)	Frequency (MHz)	Time (ms)
key_gen	[31]*	h=5,w=8	190660/126656/17	285	0.748
	This work	h=5,w=8	139886/103914/58	267	0.833
sig_gen	[23]#	h=5,w=8	-	168	1685
	This work	h=5,w=8	139886/103914/58	267	0.026
verify	[28]**	h=20,w=8	2663/1585/1	250	5
	This work	h=20,w=8	129990/68610/32.5	294	0.026

* [31] is a SHA3-based key generation acceleration work on Xilinx ZCU104 FPGA platform.
\# [23] is a SHAKE256-based signature generation acceleration work on STM32 ARM-M4 platform.
** [28] is a SHA256-based signature verification acceleration work on Xilinx XCVU7P FPGA platform.

As shown in Table X, all the state-of-the-art implementations for these three stages of LMS are compared with those of our design respectively. For the *key generation* stage, our work is about 11% slower than the work in [31] while the resource cost is reduced by 26.7% in terms of LUT. The hash modules in [31] and this work are the same. However, as mentioned in Section V-C, the clock frequency of the *Key_gen&Sign* module in this work is limited by the user clock of the DDR4 IP. Additionally, the reuse of the hash group leads to more complicated control logic than [31], which causes the calculation latency overhead. These two reasons cause the slightly lower runtime performance of the key generation in this work. For the *signature generation*, the

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
COMPARISON OF EXISTING WORKS ON IMPLEMENTATION OF LMS AND XMSS

Parameters	Design	Hash type	Platform	Frequency (MHz)	key_gen (CCs)	sig_gen (CCs)	verify (CCs)
h=5,w=8	[23]	SHA256	STM32 ARM-M4	168	9.41×10^8	8.40×10^8#	2.32×10^8
	[23]	SHAKE256	STM32 ARM-M4	168	3.88×10^9	2.83×10^8#	9.24×10^7
	[31]	SHA3-256	ZCU104 FPGA	285	2.14×10^5	-	-
	this work	SHAKE256	XCVU7P FPGA	267/294*	2.22×10^5	7049	7327
h=5,w=4	[23]	SHA256	STM32 ARM-M4	168	1.18×10^8	8.53×10^6#	2.58×10^6
	[23]	SHAKE256	STM32 ARM-M4	168	4.83×10^8	3.63×10^7#	9.24×10^7
	[31]	SHA3-256	ZCU104 FPGA	285	2.77×10^4	-	-
	this work	SHAKE256	XCVU7P FPGA	267/294*	2.82×10^4	998	1553
h=20,w=8	[28]	SHA256	XCKU025 FPGA	250	-	-	1.25×10^6
	[38]	SHA256	Intel Xeon CPU	3300	5.41×10^{11}	-	-
	this work	SHAKE256	XCVU7P FPGA	267/294*	7.26×10^9	7049	7727
h=20,w=4	[28]	SHA256	XCKU025 FPGA	250	-	-	2.00×10^5
	[38]	SHA256	Intel Xeon CPU	3300	7.10×10^{10}	-	-
	[26]**	SHA256(XMSS)	Murax Soc	93	3.30×10^{11}	1.00×10^6	5.80×10^5
	[29]**	SHA256(XMSS)	Artix-7 FPGA	110	3.27×10^{11}	7.52×10^5	1.91×10^5
	this work	SHAKE256	XCVU7P FPGA	267/294*	9.15×10^8	998	1953

* 267MHz is the maximum clock frequency of the *Key_gen&Sign* module, and 294MHz is the maximum clock frequency of the *Signature Verification* module.
The signature generation time of [23] is computed by subtracting the original signature generation time with the key generation time, which can be regarded as the pure signing time similar to the situation of our design.
** [26] and [29] are two state-of-the-art implementation woks of XMSS. [26] is a software-hardware co-design based on RISC-V. [29] is a full hardware implementation and is the fastest XMSS implementation up to now.

only available SHAKE256-based LMS implementation work is the software implementation on ARM Cortex-M4 platform in [23], which is far slower than our design. [28] proposed a SHA256-based *signature verification* acceleration work on Xilinx FPGA platform. Although it dose not have an outstanding throughput, the resource cost of the work in [28] is very low. Therefore, we use the $\frac{1}{LUT \times time}$ to evaluate the efficiency of designs. In this way, the efficiency of our work is 3.9x better than the design in [28], while our execution speed is 192x faster.

VII. CONCLUSION AND FUTURE WORK

In this paper, we extract the coarse-grained basic logic, a hash group, and carefully design the parallelism through some mathematical transformations. Base on this, a fusion architecture for *key generation* and *signature generation* is devised by reusing the hash group in these two stages. Moreover, for the first time, we propose a full hardware design for LMS and compare it with the state-of-the-art implementations. To our best knowledge, LMS has not been applied in practical applications due to its high complexity and low speed. However, our design shows a similar signature performance and stronger security strength than the practical product [39] developed by Intel for data centers. Our work can significantly promote the practical applications of LMS. Our future work is to explore more efficient control logic for the fusion architecture to further improve the *key generation* phase. And the hash group module is also being considered to be optimized in resource expansion for better area efficiency.

REFERENCES

[1] A. Cross, "The IBM Q experience and QISKit open-source quantum computing software," in *Proc. APS March Meeting Abstr.*, vol. 2018, 2018, p. L58-003.
[2] C. Monroe, "IonQ quantum computers: Clear to scale," in *Proc. APS March Meeting Abstr.*, vol. 2021, 2021, p. P10-002.
[3] Rigetti Tech. *Homepage*. [Online]. Available: https://www.rigetti.com/
[4] S.-Y. Hou *et al.*, "SpinQ gemini: A desktop quantum computer for education and research," 2021, *arXiv:2101.10017*.
[5] D. Bernstein and T. Lange, "Post-quantum cryptography," *Nature*, vol. 549, no. 7671, pp. 188–194, 2017.
[6] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Rev.*, vol. 41, no. 2, pp. 303–332, 1999.
[7] D. Moody *et al.*, "Status report on the second round of the NIST post-quantum cryptography standardization process," NIST Interagency, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Internal Rep. NISTIR, 2020. Accessed: Sep. 28, 2022, doi: 10.6028/NIST.IR.8309.
[8] D. Apon and J. Howe, "Attacks on NIST PQC 3rd round candidates," in *Proc. IACR Real World Crypto Symp.*, 2021, pp. 1–26.
[9] D. McGrew, M. Curcio, and S. Fluhrer, *Leighton–Micali Hash-Based Signatures*, RFC 8554, Apr. 2019.
[10] A. Huelsing, D. Butin, S.-L. Gazdag, J. Rijneveld, and A. Mohaisen, *XMSS: Extended Merkle Signature Scheme*, RFC 8391, May 2018.
[11] D. A. Cooper *et al.*, *Recommendation for Stateful Hash-Based Signature Schemes*, Standard NIST SP 800:208, 2020.
[12] S. Fluhrer, D. McGrew, P. Kampanakis, and V. Smyslov, "Postquantum preshared keys for IKEv2," Internet-Draft draft-ietf-ipsecme-qr-ikev2-08, Internet Eng. Task Force, Fremont, CA, USA, Tech. Rep., 2019.
[13] A. Hülsing, J. Rijneveld, S. Samardjiska, and P. Schwabe, "From 5-pass MQ-based identification to MQ-based signatures," *IACR Cryptol. ePrint Arch.*, vol. 2016, p. 708, Jun. 2016.
[14] M. Ounsworth and M. Pala, "Composite keys and signatures for use in internet PKI," Internet-Draft draft-ounsworth-pq-composite-sigs-01, Internet Eng. Task Force, Fremont, CA, USA, Tech. Rep., 2019.
[15] D. Steblia, S. Fluhrer, and S. Gueron, "Design issues for hybrid key exchange in TLS 1.3," Internet-Draft draft-stebila-tls-hybrid-design-01, Internet Eng. Task Force, Fremont, CA, USA, Tech. Rep., 2019.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.

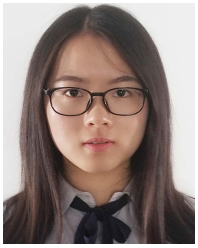
[16] C. Tjhai *et al.*, "Framework to integrate post-quantum key exchanges into internet key exchange protocol version 2 (IKEv2)," Internet-Draft draft-tjhai-ipsecme-hybrid-qske-ikev2-03, Internet Eng. Task Force, Fremont, CA, USA, Tech. Rep., 2019.
[17] C.-L. Chan, R. Fontugne, K. Cho, and S. Goto, "Monitoring TLS adoption using backbone and edge traffic," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Apr. 2018, pp. 208–213.
[18] D. Naylor *et al.*, "The cost of the 's' in https," in *Proc. 10th ACM Int. Conf. Emerg. Netw. Exp. Technol.*, 2014, pp. 133–140.
[19] HTTP Archive. *Webreport*. [Online]. Available: https://httparchive.org/reports/state-of-the-web
[20] D. Butin, S.-L. Gazdag, and J. Buchmann, "Real-world post-quantum digital signatures," in *Cyber Secur. Privacy Forum*. Cham, Switzerland: Springer, 2015, pp. 41–52.
[21] J. W. Bos, A. Hülsing, J. Renes, and C. V. Vredendaal, "Rapidly verifiable XMSS signatures," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2021, no. 1, pp. 137–168, 2021.
[22] J. Buchmann, E. Dahmen, and A. Hülsing, "XMSS—A practical forward secure signature scheme based on minimal security assumptions," in *Proc. Int. Workshop Post-Quantum Cryptogr.* Berlin, Germany: Springer, 2011, pp. 117–129.
[23] F. Campos, T. Kohlstadt, S. Reith, and M. Stöttinger, "LMS vs XMSS: Comparison of stateful hash-based signature schemes on ARM Cortex-M4," in *Proc. Int. Conf. Cryptol. Afr.* Cham, Switzerland: Springer, 2020, pp. 258–277.
[24] A. Hülsing, C. Busold, and J. Buchmann, "Forward secure signatures on smart cards," in *Proc. Int. Conf. Sel. Areas Cryptogr.* Berlin, Germany: Springer, 2012, pp. 66–80.
[25] S. Ghosh, R. Misoczki, and M. R. Sastry, "Lightweight post-quantum-secure digital signature approach for IoT motes," *IACR Cryptol. ePrint Arch.*, vol. 2019, p. 122, 2019.
[26] W. Wang *et al.*, "XMSS and embedded systems," in *Proc. Int. Conf. Sel. Areas Cryptogr.* Cham, Switzerland: Springer, 2019, pp. 523–550.
[27] V. B. Y. Kumar, N. Gupta, A. Chattopadhyay, M. Kasper, C. Krauß, and R. Niederhagen, "Post-quantum secure boot," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2020, pp. 1582–1585.
[28] P. Kampanakis, P. Panburana, M. Curcio, and C. Shroff, "Post-quantum hash-based signatures for secure boot," in *Proc. Silicon Valley Cybersec. Conf.* Springer, 2020, pp. 71–86.
[29] Y. Cao *et al.*, "An efficient full hardware implementation of extended Merkle signature scheme," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 2, pp. 682–693, Feb. 2022.
[30] P. Kampanakis and S. Fluhrer, "LMS vs XMSS: Comparion of two hash-based signature standards," *Cryptol. ePrint Arch.*, 2017.
[31] Y. Song, X. Hu, W. Wang, J. Tian, and Z. Wang, "High-speed and scalable FPGA implementation of the key generation for the Leighton–Micali signature protocol," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5.
[32] R. C. Merkle, "A certified digital signature," in *Proc. Conf. Theory Appl. Cryptol.* Springer, 1989, pp. 218–238.
[33] S. M. Radack, "Secure hash standard: Updated specifications approved and issued as federal information processing standard (FIPS) 180-4," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Tech. Rep. FIPS180-4, 2012.
[34] M. J. Dworkin, "SHA-3 standard: Permutation-based hash and extendable-output functions," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Tech. Rep. FIPS202, 2015.
[35] M. M. Wong, J. Haj-Yahya, S. Sau, and A. Chattopadhyay, "A new high throughput and area efficient SHA-3 implementation," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2018, pp. 1–5.
[36] M. M. Wong, V. Pudi, and A. Chattopadhyay, "Lightweight and high performance SHA-256 using architectural folding and 4–2 adder compressor," in *Proc. IFIP/IEEE Int. Conf. Very Large Scale Integr. (VLSI-SoC)*, Oct. 2018, pp. 95–100.
[37] H. E. Michail, G. S. Athanasiou, V. Kelefouras, G. Theodoridis, and A. C. E. Goutis, "On the exploitation of a high-throughput SHA-256 FPGA design for HMAC," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 5, no. 1, pp. 1–28, 2012.
[38] S. Fluhrer. *LMS Hash Based Signature Open-Source Implementation*. [Online]. Available: https://github.com/cisco/hash-sigs
[39] Intel Tech. *Product Brief Intel Quickassist Adapter 8950*. [Online]. Available: https://www.intel.com/content/dam/www/public/us/en/documents/product-briefs/quickassist-adapter-8950-brief.pdf



Yifeng Song received the B.S. degree in physics from Nanjing University, Nanjing, China, in 2018, where he is currently pursuing the Ph.D. degree in electronic science and technology. His research interests include VLSI design for cryptographic engineering.



Xiao Hu received the B.S. degree in information engineering from Xi'an Jiaotong University, Xi'an, China, in 2019. She is currently pursuing the Ph.D. degree in information and communication engineering with Nanjing University. Her research interests include VLSI design for cryptographic engineering.



Jing Tian (Member, IEEE) received the B.S. degree in microelectronics and the Ph.D. degree in information and communication engineering from Nanjing University, Nanjing, China, in 2015 and 2020, respectively. She is currently an Associate Researcher with Nanjing University. Her research interests include VLSI design for digital signal processing and cryptographic engineering.



Zhongfeng Wang (Fellow, IEEE) received the B.S. and M.S. degrees from Tsinghua University and the Ph.D. degree from the University of Minnesota, Minneapolis, in 2000.

Since 2016, he has been working at Nanjing University, China, as a Distinguished Professor. Previously, he worked at Broadcom Corporation, CA, USA, from 2007 to 2016, as a leading VLSI Architect. Before that, he worked at Oregon State University and National Semiconductor Corporation. He is currently a World-Recognized Expert in low-power high-speed VLSI design for signal processing systems. He has published over 200 technical papers with multiple best paper awards received from the IEEE technical societies, among which is the IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS Best Paper Award in 2007. He has edited one book VLSI and held more than 20 U.S. and Chinese patents. His current research interests are in the area of optimized VLSI design for digital communications and deep learning. He has also served as a TPC member and the various chair for ten of international conferences. Moreover, he has contributed significantly to the industrial standards. So far, his technical proposals have been adopted by more than 15 international networking standards. In 2015, he was elevated to the fellow of IEEE for contributions to VLSI design and implementation of FEC coding. In the current record, he has had many papers ranking among top 25 most (annually) downloaded manuscripts in IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS. In the past, he has served as an Associate Editor for IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS, and IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS for many terms.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on October 13,2026 at 07:59:04 UTC from IEEE Xplore. Restrictions apply.